



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Name: Akshat Kumar

Registration No: 24BCE10612

Title: LibreOffice: When Open Source Meets a Crisis of Trust

LibreOffice: When Open Source Meets a Crisis of Trust

Section A: Understanding the Situation

The story of LibreOffice is not just about software development; it is about how trust, control, and community interact in open source systems. At first, OpenOffice.org seemed like a perfect example of successful open source software. It was widely used, had strong community support, and provided a free alternative to Microsoft Office.

OpenOffice began when Sun Microsystems acquired StarOffice and released it as open source. This decision made a big difference, especially for users who could not afford expensive software. Governments, schools, and individuals started adopting it because it was free and worked across multiple platforms.

However, despite being open source, OpenOffice had a hidden issue. The project was not truly controlled by the community. Sun Microsystems had the final authority over important decisions. Contributors had to sign agreements that gave Sun shared rights over their work. This meant that even though developers contributed voluntarily, they did not fully own what they created.

Another important issue was decision-making. Sun controlled the release schedule, features, and direction of the project. Community members could suggest ideas, but they did not have real power.

For a long time, this situation continued without major problems because Sun was seen as a relatively friendly company. Developers trusted that Sun would act responsibly.

The turning point came in 2010 when Oracle acquired Sun Microsystems. This changed the entire situation. Oracle had a reputation for being more aggressive in business and more focused on profit.

After the acquisition, developers started feeling uncertain. Even though Oracle did not immediately change the license, it made decisions that reduced trust. It continued the copyright policy, introduced a commercial version of OpenOffice, and communicated less with the community.

At this point, the problem became clear. The project was open in terms of code, but not in terms of control.

This case shows the difference between:

- Open code → anyone can use it
- Open governance → everyone can influence it

OpenOffice had the first, but not the second.

OpenOffice was “open” only in **code**, not in **decision-making**

Contributors had:

- No voting power
- No roadmap control
- No release authority

“Single point of control risk”

- If one company controls everything → project stability depends on that company
- When Sun was trusted → system worked
- When Oracle came → same system became dangerous

Example idea:

- A truly open project = like democracy
- OpenOffice = like monarchy with public participation

The fork was not sudden

It was a **delayed reaction to a long-standing structural flaw**

Section B: Examining the Ethics of Control

The central ethical issue in this case is whether a company should have strong control over an open source project.

On one hand, companies like Sun and Oracle invest money, infrastructure, and professional developers. Without this support, projects might struggle to grow. From this perspective, it seems reasonable for companies to expect some level of control.

However, open source is not just about companies. It is also about individuals who contribute voluntarily. These contributors spend time and effort without direct payment. They do this because they believe in collaboration and open sharing.

The problem arises when companies gain more benefits than contributors.

The copyright assignment system is a good example. Developers contributed code, but companies gained legal rights over that code. This creates an imbalance.

Under Sun, contributors accepted this because they trusted the company. But under Oracle, the same system felt risky. This shows that ethics depend on both structure and trust.

In my opinion, companies should not fully control open source projects. Instead, they should act as partners.

There is a clear difference between:

- **Control** → making all decisions
- **Participation** → contributing while sharing power

Open source should focus on participation.

Another issue is the commercialization of community work. Oracle used OpenOffice to create a commercial product. While this is not wrong, it raises concerns if the community does not benefit equally.

A fair system should ensure that:

- Contributors keep ownership
- Decisions are shared
- Benefits are distributed

Without these, open source can become unfair.

Companies invest:

- Infrastructure (servers, tools)
- Paid developers
- Legal protection

Without companies:

- Large-scale projects may fail.

“Control can be seen as a return on investment rather than exploitation.”

Volunteers are not employees

They are not paid → yet their work is owned

“Value extraction imbalance”

- Community creates value
- Company captures value

If contributors leave → company loses everything

So real power is with community, not company.

Open source development creates a unique situation where value is generated collectively but often controlled centrally. In the case of OpenOffice.org, thousands of contributors invested their time, skills, and creativity into improving the software. However, the benefits of this collective effort were not distributed equally. While the community created the value, the controlling company—first Sun and later Oracle—had the authority to commercialize, package, and monetize that value. This creates what can be described as a “value capture imbalance,” where those who contribute the most are not necessarily those who benefit the most.

Section C: Evaluating the Decision to Fork

Forking OpenOffice to create LibreOffice was a major turning point. It was not just a technical decision, but also a social and ethical one.

Forking means taking the existing code and starting a new project independently. While this is allowed in open source, it is usually avoided because it divides the community.

In this case, the community felt that staying under Oracle's control was not a good option. They wanted a system where they could participate in decision-making.

The creation of The Document Foundation provided this opportunity. It offered a governance model that was independent and transparent.

The community even gave Oracle a chance to join this new system, but Oracle refused.

This refusal made the situation more clear. Developers had to choose between two paths:

- Stay with Oracle and accept its control
- Move to LibreOffice and build something new

Many developers chose LibreOffice because they valued independence.

However, the fork had several disadvantages:

- It split the developer community
- It confused users
- It required rebuilding trust

Some people might argue that the community should have stayed and tried to improve OpenOffice from within.

This argument has some strength. Staying would have avoided division and maintained stability.

However, it assumes that Oracle would allow meaningful change. Given the power imbalance, this seems unlikely.

In my opinion, the fork was the correct decision. It allowed the community to create a system that aligned with their values.

In the long term, LibreOffice became more successful than OpenOffice, which shows that the decision worked.

Community could have:

- Negotiated governance changes
- Created advisory boards
- Reduced dependence on Oracle

These options were **unrealistic due to power imbalance**

Business vs Ethical Perspective

Oracle (Business view):

- Control = revenue
- Brand = asset
- Fork = loss of control

So:

Business decision = logical

Ethical view:

- Ignored community
- Reduced transparency

So:

Ethical decision = weak

Fork caused:

- Confusion
- Duplication of effort
- Resource wastage

“Forking solves governance problems but creates ecosystem instability.”

Section D: Lessons and Broader Understanding

The LibreOffice case provides several important lessons for open source projects.

1. Trust is essential

Even strong projects can fail if trust is lost. Once developers lose confidence in leadership, they may leave.

2. Governance is as important as code

Open source is not just about software. It is also about how decisions are made.

3. Licensing is not enough

Even if code is open, control can still be centralized. True openness requires shared governance.

4. Community matters

Projects succeed when contributors feel respected and involved.

5. Forking is powerful but costly

Forking can solve problems, but it comes with risks. It should be used carefully.

This case also connects to other examples like MariaDB, where similar issues led to a fork.

If I were designing a new open source project, I would recommend:

- No copyright transfer
- Independent foundation
- Transparent processes
- Equal participation

These steps can help prevent conflicts.

Same pattern:

- Corporate acquisition

- Trust loss
- Fork creation

“This is not a coincidence, but a structural pattern.”

If company controls:

- Code + governance + ecosystem
→ It is **open source in name, but proprietary in practice**

Trust criteria:

- Can community reject decisions?
- Are decisions transparent?
- Is control distributed?

Section E: Personal Reflection

Before studying this case, I thought open source simply meant free software. I did not think much about governance or control.

Now I understand that open source is much more complex. It involves trust, fairness, and collaboration.

One thing that surprised me was how contributors could lose ownership of their work. This feels unfair, especially when they are not paid

.

I also found the idea of forking very interesting. It shows that open source communities have the power to Take control when needed.

The decision to create LibreOffice was risky, but it showed strong values. The community chose independence over convenience.

I believe this was the right decision. LibreOffice's success proves that trust and fairness matter in the long run.

If I were part of that community, I think I would have supported the fork. Even though it would be difficult, it would ensure a better future.

Before studying this case, my understanding of open source was quite limited. I mainly saw it as software that is free to use and accessible to everyone. I believed that as long as the source code was open, the system was fair and beneficial for all users and developers. However, this case study has significantly changed my perspective.

I now understand that open source is not just about access to code—it is also about control, governance, and trust. A project can be legally open, but still practically controlled by a single entity. This realization was surprising to me because I had never considered the difference between “open code” and “open governance.” The LibreOffice case clearly shows that true openness requires both.

One aspect that I found particularly troubling was the issue of copyright assignment. Contributors were voluntarily giving their time and effort, yet they were required to share legal ownership of their work with a corporation. While this may seem acceptable under a trusted organization, it becomes problematic when control shifts to a company with different values. This made me question whether such systems are truly fair, especially when contributors are not compensated but still lose control over their contributions.

Another concept that stood out to me was the idea of forking as a form of resistance. Before this, I saw forking as just a technical feature of open source. Now, I see it as a powerful tool that allows communities to protect their values and independence. The decision to create LibreOffice was not easy—it involved risk, uncertainty, and loss of stability. However, it also demonstrated that communities are not powerless. When governance becomes unfair, they have the ability to take control of their future.

THANKYOU